

List Based Application Project

Introduction

This project is a test to demonstrate our ability to create an object-oriented program with the ability to dynamically create, store, and update client information while handling user input and any possible errors that may create.

Implementation

To start, I first create a node class to hold the inputted data. This is the Client class. Because its only job is to store and modify the data of a single node, it's a very simple class which only needs to meet the minimum requirements of encapsulation. I included the getters, setters, next, and a toString method. This is all the node needs to function.

```
class Client
{
private:
    unsigned int accountID;
    std::string name;
    float balance;
    Client* next;

public:
    Client();
    Client(int id, std::string clientName, float initialBalance);

    // getters
    int getAccountID();
    std::string getName();
    float getBalance();
    Client* getNext();

    // setters
    void setAccountID(int id);
    void setName(std::string clientName);
    void setBalance(float newBalance);
    void setNext(Client* nextClient);

    // toString method
    std::string toString();
};
```

After I created the implementation for this class, I designed a function in the main file to gather the user input and return a new Client object, which allowed me to test the functionality of the class itself.

```
Enter account ID (positive integer): -1
Invalid account ID. Please try again.
Enter account ID (positive integer): 1
Enter client name: Testing 123
Enter initial balance: 456
Account ID: 1, Name: Testing, Balance: $456.000000
```

Shown in this screenshot is an example of error handling. First is the account ID number. In this program, users cannot have negative account ID numbers. In the first input, the user enters a negative number which is quickly discarded and replaced by the users next input after the program re-prompts for input. Then numbers are entered into the string after a space. Without error handling, the space would have overflowed into the next field and created an error. Note that even if the float value can store a value of 123, the input from the string should not overflow into the float value.

The next step is to create a second object which is responsible for storing multiple of the Client objects. It needs to have methods to add, edit, delete, and sort the Client objects. I chose to have the list sorted by ID during the add method, which will simplify the print method for IDs later.

```
// Add client - will sort clients as they are added with insertion sort.
void ClientList::addClient(Client* newClient)
{
    if (head == nullptr || newClient->getAccountID() < head->getAccountID())
    {
        newClient->setNext(head);
        head = newClient;
        return;
    }
    Client* current = head;
    while (current->getNext() != nullptr && current->getNext()->getAccountID() < newClient->getAccountID())
    {
        current = current->getNext();
    }
    newClient->setNext(current->getNext());
    current->setNext(newClient);
}
```

Shown below is the print method, simplified by the insertion sorting of the add method.

```

// Print all clients ordered by ID number. Does not modify the original list since it is already sorted by ID.
std::string ClientList::getAllClientsbyID()
{
    // if list is empty
    if (head == nullptr)
    {
        return "No clients in the list.";
    }

    // compile string of all clients
    std::string result;
    Client* current = head;
    while (current != nullptr)
    {
        result += current->toString();
        current = current->getNext();
    }
    return result;
}

```

The other sorting methods are more difficult to implement since they need to sort the list for a different value. To do this, I create a copy of the list and use insertion sort to add all values accordingly. Then I create a return string with all the node values and send it back to main.

```

// create a copy of the list to sort
Client* sortedHead = nullptr;
Client* current = head;

// Insertion sort by name
while (current != nullptr)
{
    Client* newClient = new Client(current->getAccountID(), current->getName(), current->getBalance());
    // Insert into sorted list
    if (sortedHead == nullptr || newClient->getName() < sortedHead->getName())
    {
        newClient->setNext(sortedHead);
        sortedHead = newClient;
    }
    else
    {
        Client* sortedCurrent = sortedHead;
        while (sortedCurrent->getNext() != nullptr && sortedCurrent->getNext()->getName() < newClient->getName())
        {
            sortedCurrent = sortedCurrent->getNext();
        }
        newClient->setNext(sortedCurrent->getNext());
        sortedCurrent->setNext(newClient);
    }
    current = current->getNext();
}

// compile string of all clients from sorted list
std::string result;
current = sortedHead;
while (current != nullptr)
{
    result += current->toString();
    Client* toDelete = current;
    current = current->getNext();
    delete toDelete; // clean up
}
return result;

```

```
Enter account ID (positive integer): 1
Enter client name: c
Enter initial balance: 2
Enter account ID (positive integer): 2
Enter client name: a
Enter initial balance: 3
Enter account ID (positive integer): 3
Enter client name: b
Enter initial balance: 1

All clients by ID:
Account ID: 1, Name: c, Balance: $2.000000
Account ID: 2, Name: a, Balance: $3.000000
Account ID: 3, Name: b, Balance: $1.000000

All clients by Name:
Account ID: 2, Name: a, Balance: $3.000000
Account ID: 3, Name: b, Balance: $1.000000
Account ID: 1, Name: c, Balance: $2.000000

All clients by Balance:
Account ID: 3, Name: b, Balance: $1.000000
Account ID: 1, Name: c, Balance: $2.000000
Account ID: 2, Name: a, Balance: $3.000000
```

Finally, all that is left after creating the ClientList is to create the program in main. First, I create a function to create the main menu and receive user input.

```

int mainMenu()
{
    int option;
    cout << "Welcome to the Client Management System!" << endl;
    cout << "1. Add Client" << endl;
    cout << "2. Delete Client" << endl;
    cout << "3. Find Client" << endl;
    cout << "4. Update Client Balance" << endl;
    cout << "5. Display All Clients by ID" << endl;
    cout << "6. Display All Clients by Name" << endl;
    cout << "7. Display All Clients by Balance" << endl;
    cout << "8. Exit" << endl;
    cout << "Please select an option (1-8): ";
    do
    {
        cin.clear(); // Clear error flag
        cin >> option;
        if (cin.fail() || option < 1 || option > 8)
        {
            cin.ignore(numeric_limits<streamsize>::max(), '\n');
            cout << "Invalid account ID. Please try again." << endl;
        }
    } while (cin.fail() || option < 1 || option > 8);

    return option;
}

```

Then all that was left was to have the function calls put into the main function to create the functionality.

```

option = mainMenu();
switch (option)
{
case 1:
{
    Client client = newClient();
    Client* clientPtr = new Client(client.getAccountID(), client.getName(), client.getBalance());
    clientList.addClient(clientPtr);
    cout << "Client added successfully." << endl;
    break;
}
case 2:
{
    if (!clientList.isEmpty())
    {
        cout << "Enter account ID to delete: ";
        cin >> accID;
        clientList.deleteClient(accID);
        cout << "Client deleted successfully." << endl;
    }
    else
    {
        cout << "Client list is empty. Cannot delete client." << endl;
    }
    break;
}
case 3:
{
    if (!clientList.isEmpty())
    {
        cout << "Enter account ID to find: ";
        cin >> accID;
        cout << clientList.findClient(accID) << endl;
    }
    else
    {
        cout << "Client list is empty. Cannot find client." << endl;
    }
    break;
}
case 4:
{

```

Conclusion

Test Cases

Unfortunately, I don't have anything interesting to say about testing. The only huge bug I had was when user input failed and did funny things with the input stream. I did my best to try and fix this issue utilizing tools such as `cin.fail()` and `cin.ignore()`, however the issue is not completely fixed. While I cannot completely break the program with bad input, it still can produce undesirable results. It's clear I need to do more research into input validation.

Challenges

I didn't have many noteworthy issues with this project. It was an evolution of the linked lists which I have done previously and my experience with those have shown me what works, and since this project was so like the previous assignments, I had a lot of reference material to base my design on. My biggest challenge was the sorting algorithms. Originally I had my program resorting the entire list every time a new sort was needed. However, I quickly realized that solution over complicated the project. I then realized I could just copy

the data from the original list and resort the data into a new one, then return the copy. This made the program much easier to handle.